

Du Chiffrement Fonctionnel pour Produits Scalaires aux Machines de Turing

Définitions, Constructions et Preuves
Cybersécurité — Chiffrement Fonctionnel

Mars 2026

Résumé

Ce document présente les définitions et preuves sous-jacentes à la présentation « De l'IPFE aux Machines de Turing ». Quatre résultats sont développés : (1) la borne inférieure algébrique de l'IPFE, (2) la difficulté de LWE et sa réduction au pire cas vers GapSVP (Regev 2005), (3) la construction FE pour MT depuis LWE (Ananth-Vaikuntanathan, CRYPTO 2021) avec preuve IND-CPA sélective par hybrid argument, et (4) la sécurité adaptative via Punctured Programming (Agrawal et al., ASIACRYPT 2022).

Chaque preuve est précédée d'une **boîte intuition** expliquant l'idée informelle avant la démonstration formelle. Les simulations Python (`ipfe_limit.py`, `fe_mt_demo.py`, `circuit_vs_tm.py`) adoptent un style procédural ; les sources de chaque construction sont détaillées dans les entêtes des scripts.

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Organisation	3
2	Définitions	3
2.1	Chiffrement à clé publique	3
2.2	Chiffrement Fonctionnel	3
2.3	Sécurité IND-CPA	3
2.4	IPFE	4
2.5	Circuit booléen	4
2.6	Machine de Turing	4
2.7	Garbled RAM	4
2.8	Hypothèse LWE	5
2.9	Pourquoi LWE est difficile	5
3	Borne inférieure de l'IPFE	6
4	FE pour MT depuis LWE	7
4.1	Théorème principal	7
4.2	Construction	7
4.3	Preuve de sécurité	8
5	Sécurité Adaptive via Punctured Programming	9
5.1	Clé percée	9
5.2	Preuve	9

6	État de l'art	10
6.1	Tableau récapitulatif	10
6.2	Registered FE pour MT (2025)	10
6.3	Frontières ouvertes	11
7	Synthèse	11

1 Introduction

1.1 Contexte

Le Chiffrement Fonctionnel (FE) généralise le chiffrement à clé publique en permettant un contrôle fin sur l'information divulguée. Plutôt que révéler le message x entier, une clé fonctionnelle sk_f ne révèle que $f(x)$.

L'IPFE (Inner-Product FE) réalise ce paradigme pour les fonctions linéaires $f_{\mathbf{y}}(\mathbf{x}) = \langle \mathbf{x}, \mathbf{y} \rangle$, avec une limitation fondamentale : la dimension n est figée au **Setup**. Les Machines de Turing brisent cette limitation : un schéma FE pour MT vérifie $\text{Dec}(sk_M, \text{Enc}(x)) = M(x)$ pour tout $x \in \{0, 1\}^*$, avec une seule clé sk_M valable quelle que soit la taille de x .

1.2 Organisation

- **Section 2** : Définitions (PKE, FE, IPFE, Circuit, MT, Garbled RAM, LWE et sa difficulté).
- **Section 3** : Borne inférieure de l'IPFE.
- **Section 4** : FE pour MT — construction et preuve de sécurité.
- **Section 5** : Sécurité adaptative via Punctured Programming.
- **Section 6** : État de l'art et problèmes ouverts.

2 Définitions

2.1 Chiffrement à clé publique

Définition 2.1 (PKE). Un *schéma PKE* est un tuple $(\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ avec :

$$\text{Setup}(1^\lambda) \rightarrow (pk, sk), \quad \text{Enc}(pk, m) \rightarrow c, \quad \text{Dec}(sk, c) \rightarrow m$$

satisfaisant la complétude $\Pr[\text{Dec}(sk, \text{Enc}(pk, m)) = m] = 1$.

Limitation. Dec révèle m entièrement ou rien. Aucune granularité sur l'information divulguée.

2.2 Chiffrement Fonctionnel

Définition 2.2 (FE, Boneh-Sahai-Waters 2011). Soit $\mathcal{F} = \{f : \mathcal{X} \rightarrow \mathcal{Y}\}$ une famille de fonctions. Un *schéma FE* pour $\mathcal{F}$ est un tuple $(\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$:

$$\begin{aligned} \text{Setup}(1^\lambda) &\rightarrow (mpk, msk) \\ \text{KeyGen}(msk, f) &\rightarrow sk_f, \quad f \in \mathcal{F} \\ \text{Enc}(mpk, x) &\rightarrow c, \quad x \in \mathcal{X} \\ \text{Dec}(sk_f, c) &\rightarrow f(x) \end{aligned}$$

Propriété fondamentale. Posséder sk_f révèle *uniquement* $f(x)$; aucune autre information sur x n'est accessible.

2.3 Sécurité IND-CPA

Définition 2.3 (IND-CPA pour le FE). Le jeu IND-CPA_λ se déroule entre $\mathcal{C}$ et $\mathcal{A}$:

1. $\mathcal{C}$ calcule $(mpk, msk) \leftarrow \text{Setup}(1^\lambda)$, envoie mpk à $\mathcal{A}$.
2. $\mathcal{A}$ demande des clés $sk_{f_1}, \dots, sk_{f_q}$ via $\text{KeyGen}(msk, \cdot)$.
3. $\mathcal{A}$ soumet $x_0, x_1 \in \mathcal{X}$ t.q. $f_i(x_0) = f_i(x_1)$ pour tout i .
4. $\mathcal{C}$ tire $b \xleftarrow{\$} \{0, 1\}$, retourne $c^* = \text{Enc}(mpk, x_b)$.

5. $\mathcal{A}$ peut continuer les requêtes (même contrainte).
6. $\mathcal{A}$ produit $b' \in \{0, 1\}$.

L'avantage est $\mathbf{Adv}_{\mathcal{A}}^{\text{FE}}(\lambda) = |\Pr[b' = b] - \frac{1}{2}|$. Le schéma est *IND-CPA sécurisé* ssi pour tout PPT $\mathcal{A}$, $\mathbf{Adv}_{\mathcal{A}}^{\text{FE}}(\lambda) \leq \text{negl}(\lambda)$.

Remarque 2.4. La contrainte $f_i(x_0) = f_i(x_1)$ est nécessaire : ce que les clés révèlent déjà ne doit pas permettre de distinguer. Sans elle, la définition est trivialement non satisfiable (prendre $f = \text{id}$).

Définition 2.5 (Sécurité sélective vs. adaptive). — **Sélective** : $\mathcal{A}$ annonce x_0, x_1 *avant* de voir mpk . Le réducteur peut programmer mpk en conséquence — modèle plus simple.

— **Adaptive** : $\mathcal{A}$ choisit x_0, x_1 *après* avoir vu mpk et obtenu des clés — modèle d'attaque réaliste.

La sécurité adaptive implique strictement la sécurité sélective.

2.4 IPFE

Définition 2.6 (IPFE). L'IPFE est un schéma FE pour :

$$\mathcal{F}_{\text{IP}}^{(n)} = \{f_{\mathbf{y}} : \mathbf{x} \mapsto \langle \mathbf{x}, \mathbf{y} \rangle \bmod p \mid \mathbf{y} \in \mathbb{Z}_p^n\}$$

où $n \in \mathbb{N}$ est la **dimension fixée au Setup** et p un premier.

2.5 Circuit booléen

Définition 2.7 (Circuit booléen). Un *circuit booléen* est un DAG dont les feuilles sont les entrées (bits), les nœuds internes des portes (AND, OR, NOT, XOR), et les racines les sorties. La taille $|C|$ et la profondeur $d(C)$ sont **figées à la construction**.

2.6 Machine de Turing

Définition 2.8 (Machine de Turing déterministe). Une MT déterministe est :

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$$

avec $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$. La description $|M|$ est une **constante indépendante de $|x|$** ; le même M traite des entrées de toute taille ; le temps $T_M(n)$ varie avec n .

Définition 2.9 (FE pour MT). Un schéma *FE pour MT* est un tuple (Setup, KeyGen, Enc, Dec) :

$$\begin{aligned} \text{Setup}(1^\lambda) &\rightarrow (mpk, msk) \\ \text{KeyGen}(msk, M) &\rightarrow sk_M \quad (M \text{ Machine de Turing}) \\ \text{Enc}(mpk, x) &\rightarrow c \quad (x \in \{0, 1\}^*, \text{ taille quelconque}) \\ \text{Dec}(sk_M, c) &\rightarrow M(x) \end{aligned}$$

Propriétés de taille : $|c| = \text{poly}(\lambda, |x|)$, $|sk_M| = \text{poly}(\lambda, |M|)$.

2.7 Garbled RAM

Définition 2.10 (Garbled RAM, Lu-Ostrovsky 2013). Un *Garbled RAM* (Garble, Eval) satisfait :

- $\text{Garble}(M, x) \rightarrow (\tilde{M}, \tilde{x})$ (brouille l'exécution).
- $\text{Eval}(\tilde{M}, \tilde{x}) \rightarrow M(x)$ (retourne uniquement le résultat).

Sécurité (simulation). Il existe $\mathcal{S}$ t.q. :

$$(\tilde{M}, \tilde{x}) \approx_c \mathcal{S}(M(x), T_M(|x|), |M|, |x|)$$

L'évaluateur n'apprend rien sur M ni x , seulement $M(x)$ et des tailles.

Remarque 2.11 (Garbled RAM vs Garbled Circuit). Un Garbled Circuit de Yao prendrait une copie du circuit pour chaque pas de la MT, ce qui donnerait une taille $T_M(n) \cdot \text{poly}(\lambda)$ — trop grand. Le Garbled RAM évalue directement une RAM (équivalente à la MT) et utilise un ORAM pour masquer le pattern d'accès mémoire : le surcoût de l'ORAM est seulement polylogarithmique en la taille de la mémoire.

2.8 Hypothèse LWE

Définition 2.12 (LWE, Regev 2005). La décision-LWE $_{n,m,q,\chi}$ est le problème de distinguer :

$$(A, As + \mathbf{e}) \approx_c (A, \mathbf{u})$$

pour $A \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$, $\mathbf{e} \xleftarrow{\chi} \mathbb{Z}^m$ (bruit gaussien discret), $\mathbf{u} \xleftarrow{\$} \mathbb{Z}_q^m$.

2.9 Pourquoi LWE est difficile

Intuition

Le rôle du bruit. L'attaquant observe A (matrice aléatoire publique) et $\mathbf{b} = As + \mathbf{e} \bmod q$, mais ne voit ni $\mathbf{s}$ ni $\mathbf{e}$. Sans le bruit, résoudre $\mathbf{b} = As \bmod q$ est trivial par élimination gaussienne. Le bruit $\mathbf{e}$ rend toutes les opérations algébriques classiques inutilisables : elles propagent et amplifient l'erreur jusqu'à obtenir un résultat inexploitable. La difficulté provient ensuite d'une connexion profonde avec la géométrie des réseaux euclidiens.

Détaillons les deux niveaux de la difficulté de LWE.

Niveau 1 : pourquoi le bruit rend l'inversion directe impossible.

Sans bruit, $\mathbf{b} = As \bmod q$ est un système linéaire sur $\mathbb{Z}_q$: l'élimination gaussienne le résout en $O(n^3)$. Avec un bruit $\mathbf{e}$ dont les entrées sont petites devant q , il faut contrôler le rapport $\|\mathbf{e}\|/q$: si $\mathbf{e}$ est trop grand, le signal est noyé dans le bruit ; s'il est trop petit, des attaques algébriques (arrondi naïf, etc.) peuvent s'appliquer. Le choix de la distribution χ (gaussienne discrète de paramètre αq , avec $\alpha = o(1/\sqrt{n} \log n)$) calibre ce compromis. Concrètement, toute tentative de manipulation linéaire de $\mathbf{b}$ pour isoler $\mathbf{s}$ fait grossir $\mathbf{e}$ jusqu'à le rendre indiscernable d'un vecteur aléatoire dans $\mathbb{Z}_q^m$.

Niveau 2 : connexion avec les réseaux euclidiens et GapSVP.

L'instance LWE $(A, \mathbf{b} = As + \mathbf{e})$ définit implicitement un *réseau euclidien* :

$$\Lambda(A) = \{A^\top \mathbf{z} \bmod q \mid \mathbf{z} \in \mathbb{Z}^m\} \subset \mathbb{Z}_q^n$$

Géométriquement, $\mathbf{s}$ est un point de $\Lambda(A)$ légèrement perturbé par $\mathbf{e}$: retrouver $\mathbf{s}$ depuis $\mathbf{b}$ revient à trouver le point du réseau *le plus proche* de $\mathbf{b}$. C'est exactement le problème *BDD* (Bounded Distance Decoding), qui est lui-même une instance du problème du vecteur le plus court :

$$\text{BDD} \subseteq \text{SVP (Shortest Vector Problem)}$$

La réduction fondamentale de Regev formalise ce lien.

Théorème 2.13 (Regev, STOC 2005). *Pour tout $\alpha \in (0, 1)$ tel que $\alpha q \geq 2\sqrt{n}$, il existe une réduction quantique en temps polynomial :*

$$\text{résoudre LWE}_{n,q,\alpha} \implies \text{résoudre GapSVP}_\gamma \text{ dans le pire cas, } \gamma = O\left(\frac{\sqrt{n}}{\alpha}\right)$$

où GapSVP_γ est le problème d'approximation du vecteur le plus court à un facteur γ .

Remarque 2.14 (Lecture de la réduction). Le sens de l'implication est : *si un algorithme efficace brise LWE , alors il en existe un qui résout GapSVP_γ sur **n'importe quel** réseau euclidien de dimension n (réduction pire cas)*. Par contraposée :

$$\text{GapSVP}_\gamma \text{ est difficile} \implies \text{LWE}_{n,q,\alpha} \text{ est difficile}$$

La sécurité de LWE repose donc sur la dureté de GapSVP dans le pire cas, et non seulement dans le cas moyen.

Remarque 2.15 (NP-dureté de SVP et de GapSVP). Le problème exact SVP est NP-difficile (et même NP-complet) en dimension finie. La preuve classique procède par réduction depuis 3-SAT : toute instance 3-SAT peut être encodée en un problème SVP en dimension $O(n)$, et cette réduction s'étend à toute dimension d fixée. (3-SAT est NP-complet par le théorème de Cook-Levin.)

Pour l'approximation, GapSVP_γ avec $\gamma = \text{poly}(n)$ est NP-difficile sous la conjecture $\text{P} \neq \text{NP}$, et les meilleures attaques connues (BKZ-2.0) restent en $2^{O(0.292 \cdot n)}$. Plus important pour la cryptographie : **aucun algorithme quantique subexponentiel** (Shor, etc.) n'est connu pour SVP ou GapSVP , ce qui fonde la résistance post-quantique de LWE .

3 Borne inférieure de l'IPFE

Intuition

Idée centrale. Le produit scalaire $\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i=1}^n x_i y_i$ n'est défini qu'entre vecteurs de même taille. Si l'on essaie de chiffrer un vecteur de dimension $n' \neq n$ avec un schéma conçu pour la dimension n , on crée une ambiguïté irréductible :

- Si $n' > n$: les composantes $x_{n+1}, \dots, x_{n'}$ n'ont pas de “coefficient partenaire” dans $\mathbf{y} \in \mathbb{Z}_p^n$.
- Si $n' < n$: les dernières composantes de $\mathbf{y}$ restent sans partenaire.

Ce n'est pas un choix d'implémentation qu'on pourrait contourner — c'est une contrainte *algébrique*. La preuve formelle montre qu'un mécanisme de conversion hypothétique permettrait à un adversaire de récupérer une composante de x en clair, cassant IND-CPA.

Théorème 3.1 (Contrainte dimensionnelle de l'IPFE). *Soit Π un schéma IPFE pour $\mathcal{F}_{\text{IP}}^{(n)}$. Pour tout $\mathbf{x}' \in \mathbb{Z}_p^{n'}$ avec $n' \neq n$, il est impossible de calculer $\langle \mathbf{x}', \mathbf{y} \rangle$ avec les clés de Π . Un nouveau Setup complet (invalidant toutes les clés) est nécessaire.*

Démonstration. La preuve s'articule en trois parties.

Partie 1 : contrainte algébrique du produit scalaire.

Rappel. Le produit scalaire $\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i=1}^n x_i y_i$ n'est défini que si $\mathbf{x}$ et $\mathbf{y}$ ont le même nombre de composantes.

Cas $n' > n$: Le vecteur $\mathbf{x}' = (x_1, \dots, x_n, x_{n+1}, \dots, x_{n'})$ possède $n' - n$ composantes supplémentaires $x_{n+1}, \dots, x_{n'}$. Aucune clé $sk_{\mathbf{y}}$ générée pour $\mathbf{y} \in \mathbb{Z}_p^n$ ne “voit” ces coordonnées ; leur contribution à $\langle \mathbf{x}', \mathbf{y} \rangle$ est structurellement indéfinie, car $\mathbf{y}$ n'a pas d'indices $n+1, \dots, n'$.

Cas $n' < n$: Le vecteur $\mathbf{y} \in \mathbb{Z}_p^n$ possède $n - n'$ composantes $y_{n'+1}, \dots, y_n$ qui n'ont aucun partenaire dans $\mathbf{x}' \in \mathbb{Z}_p^{n'}$. La valeur $\langle \mathbf{x}', \mathbf{y} \rangle$ est ambiguë : elle devrait être $\sum_{i=1}^{n'} x'_i y_i$, mais $sk_{\mathbf{y}}$ encode toujours $\langle msk, \mathbf{y}, \cdot \rangle_{\mathbb{Z}_p}$ pour $\mathbf{y}$ de taille n , sans moyen de distinguer les $n - n'$ composantes excédentaires.

Partie 2 : la clé maîtresse encode la dimension.

Dans le schéma d'Abdalla et al. (DDH), la clé maîtresse est $msk = \mathbf{s} \in \mathbb{Z}_p^n$. Le chiffrement produit :

$$\text{Enc}(mpk, \mathbf{x}) = (g^r, g^{x_1 + rs_1}, \dots, g^{x_n + rs_n}) \in \mathbb{G}^{n+1}$$

C'est un $(n + 1)$ -uplet : sa structure même révèle et impose la dimension n . Un chiffré pour $\mathbf{x}' \in \mathbb{Z}_p^{n'}$ serait un $(n' + 1)$ -uplet, incompatible avec le déchiffrement conçu pour la dimension n .

Simulation (script `ipfe_limit.py`) : `Enc` retourne un couple $(r, [c_1, \dots, c_n])$ où $c_i = x_i + r \cdot s_i \bmod p$. `Dec` calcule $\sum c_i y_i - r \cdot sk_y \bmod p$, ce qui donne bien $\sum x_i y_i$ car r s'annule algébriquement. Un appel `enc(s, [12, 15, 9, 7, 3])` lève une `ValueError` non pas par défaut de programmation, mais parce que le schéma ne peut former qu'un $(n + 1)$ -uplet structuré autour des n composantes de msk .

Le même raisonnement vaut pour LWE : `Enc` encode les n composantes via une matrice $A \in \mathbb{Z}_q^{m \times n}$; une matrice $A' \in \mathbb{Z}_q^{m \times n'}$ est structurellement incompatible.

Partie 3 : invalidation des clés (par l'absurde).

Hypothèse. Supposons qu'il existe un mécanisme de conversion ϕ tel que $\phi(\text{Enc}(mpk, \mathbf{x}'))$ soit déchiffable par les clés de dimension n .

Construction de l'adversaire. Posons $\mathbf{x}' = (\mathbf{x}_0, z)$ où $\mathbf{x}_0 \in \mathbb{Z}_p^n$ est fixé et $z \in \mathbb{Z}_p$ est une valeur secrète (la $(n + 1)$ -ième coordonnée). Soit $\mathbf{e}_{n+1}$ le vecteur unité sélectionnant la $(n + 1)$ -ième composante.

L'adversaire $\mathcal{A}$ demande la clé $sk_{\mathbf{e}_{n+1}}$. Par l'hypothèse, ϕ fonctionne, donc :

$$\text{Dec}(sk_{\mathbf{e}_{n+1}}, \phi(\text{Enc}(mpk, \mathbf{x}'))) = \langle \mathbf{x}', \mathbf{e}_{n+1} \rangle = z$$

Ainsi $\mathcal{A}$ obtient z en clair depuis le chiffré de $\mathbf{x}' = (\mathbf{x}_0, z)$, en violation directe de la sécurité IND-CPA (la composante z n'est révélée par aucune clé fonctionnelle).

Conclusion. ϕ ne peut exister. La seule option est d'exécuter `Setup`($1^\lambda, n'$) avec la nouvelle dimension, produisant un msk' indépendant de msk , et invalidant toutes les clés de l'ancien schéma. $\square$

Corollaire 3.2 (Inexpressivité pour les programmes à entrée variable). *L'IPFE ne peut pas exprimer, en une seule instance, tout programme P acceptant des entrées de taille variable (ex. : tri, comptage, analyse syntaxique sur des flux de longueur quelconque).*

Remarque 3.3 (Circuits booléens et même limitation). Un circuit C de taille T et n entrées ne peut évaluer que des entrées de n bits. Pour des entrées de taille jusqu'à N , il faut un circuit du pire cas de taille $O(N \cdot \text{cost}(f))$, croissant **linéairement** avec N .

La différence fondamentale est celle entre calcul *non-uniforme* (circuits : description différente pour chaque taille n) et calcul *uniforme* (MT : même description M pour tout n). C'est cette uniformité qui permet $|sk_M| = \text{poly}(\lambda, |M|)$ indépendamment de $|x|$ dans le FE pour MT.

4 FE pour MT depuis LWE

4.1 Théorème principal

Théorème 4.1 (Ananth-Vaikuntanathan, CRYPTO 2021). *Sous l'hypothèse LWE et pour une collusion bornée $q = \text{poly}(\lambda)$, il existe un schéma FE pour MT (Déf. 2.9) satisfaisant la sécurité IND-CPA sélective, avec :*

$$|c| = \text{poly}(\lambda, |x|), \quad |sk_M| = \text{poly}(\lambda, |M|), \quad T_{\text{Dec}} = \text{poly}(\lambda, T_M(|x|))$$

4.2 Construction

Construction 4.2 (FE/MT depuis IBE-LWE et Garbled RAM). On utilise un schéma IBE depuis LWE (GPV 2008) et un Garbled RAM (Lu-Ostrovsky 2013).

`Setup`(1^λ) : exécuter $(mpk_{\text{IBE}}, msk_{\text{IBE}}) \leftarrow \text{IBE.Setup}(1^\lambda)$ et poser $(mpk, msk) = (mpk_{\text{IBE}}, msk_{\text{IBE}})$.

$\text{KeyGen}(msk, M)$: générer une clé IBE pour l'identité $\text{id}(M) = H(M)$ (hash de la description de M) :

$$sk_M \leftarrow \text{IBE.KeyGen}(msk_{\text{IBE}}, \text{id}(M))$$

Pourquoi utiliser $H(M)$ comme identité ? L'IBE lie chaque clé à une identité publique. En prenant $H(M)$, on s'assure que sk_M est dérivé spécifiquement pour M : le déchiffrement ne fonctionnera que si la clé IBE présentée correspond bien à la MT M ayant produit ce chiffré. Taille : $|sk_M| = \text{poly}(\lambda, |M|)$, indépendante de $|x|$.

$\text{Enc}(mpk, x)$: choisir $r \xleftarrow{\$} \{0, 1\}^\lambda$ et calculer :

$$(\tilde{M}_{\text{univ}}, \tilde{x}) \leftarrow \text{Garble}(M_{\text{univ}}, x; r)$$

Le chiffré $c = (\tilde{M}_{\text{univ}}, \text{IBE.Enc}(mpk_{\text{IBE}}, K_r))$ où K_r est la clé de débrouillage du Garbled RAM liée à r . Pour tout M , $\tilde{x}$ encapsule une clé IBE pour $\text{id}(M)$. Taille : $|c| = \text{poly}(\lambda, |x|)$.

$\text{Dec}(sk_M, c)$:

1. Déchiffrer IBE pour obtenir K_r via sk_M .
2. Calculer $\text{Eval}(\tilde{M}_{\text{univ}}, \tilde{x}) \rightarrow M(x)$.

Pourquoi cela marche ? sk_M est la clé IBE pour $\text{id}(M)$, et le chiffré contient $\text{IBE.Enc}(mpk, K_r)$ lié à cette identité. Le Garbled RAM donne ensuite $M(x)$ sans révéler x directement.

4.3 Preuve de sécurité

Intuition

Stratégie générale. L'adversaire sélectif connaît x_0, x_1 avant de voir mpk , et obtient q clés $sk_{M_1}, \dots, sk_{M_q}$ avec $M_i(x_0) = M_i(x_1)$.

L'idée est de remplacer les clés une par une par des *clés simulées* : des clés qui produisent les bons résultats $M_i(x_0) = M_i(x_1)$ sans avoir besoin de voir x_b en clair. À chaque étape on change une clé, et on montre que le jeu ne change pas (computationnellement) pour l'adversaire. Au dernier jeu (H_q), toutes les clés sont simulées et le chiffré est $\text{Enc}(x_0)$ — l'adversaire ne peut plus deviner b .

C'est le **hybrid argument** : si l'adversaire avantage notablement entre H_0 et H_q , il existe un indice k où H_{k-1} et H_k sont distinguables, ce qui contredit la sécurité du Garbled RAM.

Preuve du Théorème 4.1 (détaillée). L'adversaire sélectif $\mathcal{A}$ annonce x_0, x_1 avant de voir mpk , demande q clés $sk_{M_1}, \dots, sk_{M_q}$ avec $M_i(x_0) = M_i(x_1)$ pour tout i , et reçoit $c^* = \text{Enc}(mpk, x_b)$.

Définition de la chaîne hybride.

Jeux hybrides $H_0, H_1, \dots, H_q$

H_0 : jeu réel. $c^* = \text{Enc}(mpk, x_b)$, $b \xleftarrow{\$} \{0, 1\}$. Toutes les clés $sk_{M_1}, \dots, sk_{M_q}$ sont générées normalement.

H_k ($1 \leq k \leq q$) : les k premières clés sont simulées. Le simulateur $\mathcal{S}_k$ génère $sk_{M_1}^{\text{sim}}, \dots, sk_{M_k}^{\text{sim}}$ qui produisent $M_i(x_0)$ sans connaître x_b . Le chiffré est modifié en $c^* = \text{Enc}(mpk, x_0)$ (indépendant de b). Les clés restantes $sk_{M_{k+1}}, \dots, sk_{M_q}$ sont normales.

Transition $H_{k-1} \rightarrow H_k$.

On remplace la clé sk_{M_k} (normale) par $sk_{M_k}^{\text{sim}}$ (simulée).

Pourquoi c'est possible ? Puisque $M_k(x_0) = M_k(x_1)$, le Garbled RAM produit la même sortie sur x_0 et x_1 pour M_k . Un simulateur peut donc générer $sk_{M_k}^{\text{sim}}$ qui évalue correctement à $M_k(x_0)$ sans connaître x_0 en clair.

Formellement, si $\mathcal{A}$ distingue H_{k-1} de H_k avec avantage ϵ , on construit un réducteur $\mathcal{B}_k$ qui distingue le Garbled RAM réel de sa simulation avec le même avantage :

$$|\Pr[\mathcal{A} \text{ gagne } H_{k-1}] - \Pr[\mathcal{A} \text{ gagne } H_k]| \leq \text{Adv}_{\mathcal{B}_k}^{\text{GRAM}}(\lambda)$$

Jeu final H_q .

Dans H_q , toutes les clés sont simulées et $c^* = \text{Enc}(mpk, x_0)$, indépendant de b . La sécurité IND-CPA de l'IBE garantit que mpk ne révèle pas x_0 , donc :

$$\Pr[\mathcal{A} \text{ gagne dans } H_q] = \frac{1}{2}$$

Bilan par télescope.

$$\text{Adv}_{\mathcal{A}}^{\text{FE}}(\lambda) = \left| \Pr[\mathcal{A} \text{ gagne } H_0] - \frac{1}{2} \right| \leq \sum_{k=1}^q \text{Adv}_{\mathcal{B}_k}^{\text{GRAM}}(\lambda) + \text{Adv}_{\mathcal{B}_{\text{IBE}}}^{\text{IBE}}(\lambda) \leq (q+1) \cdot \text{negl}(\lambda) = \text{negl}(\lambda)$$

La dernière inégalité tient car LWE implique la sécurité du Garbled RAM (via OWF et PRF) et la sécurité IBE-GPV. L'inégalité est valide pour $q = \text{poly}(\lambda)$. $\square$

5 Sécurité Adaptive via Punctured Programming

Intuition

Pourquoi la preuve sélective ne suffit pas. Dans la preuve sélective, le réducteur connaît x_0, x_1 avant de construire mpk , et peut “programmer” mpk de façon à ce que la simulation fonctionne. En mode adaptatif, x_0, x_1 sont choisis *après* que $\mathcal{A}$ a vu mpk — le réducteur ne peut pas anticiper.

Solution : clé percée $msk[x^*]$. On crée une version “trouée” de msk qui fonctionne normalement pour tous les inputs sauf x^* , et qui en x^* ne révèle pas $f(x^*)$. Si $f(x^*) = f(x')$ pour un x' connu, la clé percée peut quand même générer sk_f correcte (la valeur $f(x^*)$ ne discrimine pas). La FHE permet d'évaluer M_k “à l'aveugle” : depuis $\text{Enc}_{\text{FHE}}(x_0)$, on calcule $\text{Enc}_{\text{FHE}}(M_k(x_0))$ sans voir x_0 en clair.

Théorème 5.1 (Agrawal-Feng-Sarkar-Yadav, ASIACRYPT 2022). *Sous $\text{LWE} + \text{IBE} + \text{FHE}$ (sans iO), pour une collusion bornée q , il existe un schéma FE pour MT satisfaisant la sécurité IND-CPA adaptive.*

5.1 Clé percée

Définition 5.2 (Clé percée). La clé percée $msk[x^*]$ satisfait :

1. Pour tout f et $x \neq x^*$: $\text{KeyGen}(msk[x^*], f)$ est correcte.
2. En x^* : $msk[x^*]$ ne révèle pas $f(x^*)$ pour aucun f .

Si de plus $f(x^*) = f(x')$ pour un x' connu, la clé percée peut tout de même générer des clés pour f : la valeur $f(x^*)$ ne discrimine pas.

5.2 Preuve

Preuve du Théorème 5.1 (structure). **Difficulté spécifique au mode adaptatif.** Dans la sécurité sélective, le réducteur connaît x_0, x_1 avant de construire mpk et peut programmer mpk pour x_0, x_1 . En mode adaptatif, x_0, x_1 sont choisis *après* mpk — le réducteur ne peut pas anticiper. C'est pourquoi la même preuve hybride ne s'applique pas directement.

Hybrid argument adaptatif.

Soient $sk_{M_1}, \dots, sk_{M_q}$ les q clés de $\mathcal{A}$, avec $M_i(x_0) = M_i(x_1)$ pour tout i .

H_0 : jeu adaptatif réel. $c^* = \text{Enc}(mpk, x_b)$, $b \xleftarrow{\$} \{0, 1\}$. Toutes les clés normales.

H_k : les clés $sk_{M_1}, \dots, sk_{M_k}$ proviennent de la clé percée $msk[x_0]$. Le chiffré est $\text{Enc}(mpk, x_0)$ (indépendant de b). Les clés restantes sont normales.

Transition $H_{k-1} \rightarrow H_k$.

On remplace sk_{M_k} par la version issue de $msk[x_0]$. Puisque $M_k(x_0) = M_k(x_1)$, la clé percée peut générer sk_{M_k} correcte pour tout $x \neq x_0$.

Rôle de la FHE : le réducteur calcule $\text{Enc}_{\text{FHE}}(M_k(x_0))$ depuis $\text{Enc}_{\text{FHE}}(x_0)$ sans voir x_0 en clair (évaluation homomorphe de M_k sur le chiffré FHE de x_0). Cela rend la simulation possible même en mode adaptatif.

L'indiscernabilité $H_{k-1} \approx_c H_k$ se réduit à la sécurité du FHE :

$$|\Pr[\mathcal{A} \text{ gagne } H_{k-1}] - \Pr[\mathcal{A} \text{ gagne } H_k]| \leq \text{Adv}_{\mathcal{B}_k}^{\text{FHE}}(\lambda)$$

Jeu final H_q .

Dans H_q , toutes les clés viennent de $msk[x_0]$ et le chiffré est $\text{Enc}(mpk, x_0)$. L'IBE assure que mpk ne révèle pas x_0 directement. L'adversaire ne peut distinguer $b = 0$ de $b = 1$: $\Pr[\mathcal{A} \text{ gagne dans } H_q] = 1/2$.

Bilan.

$$\text{Adv}_{\mathcal{A}}^{\text{FE}}(\lambda) \leq \sum_{k=1}^q \text{Adv}_{\mathcal{B}_k}^{\text{FHE}}(\lambda) + \text{Adv}_{\mathcal{B}_{\text{IBE}}}^{\text{IBE}}(\lambda) \leq (q+1) \cdot \text{negl}(\lambda) = \text{negl}(\lambda)$$

□

Remarque 5.3 (Pourquoi pas iO ?). Pour les circuits généraux, le passage sélectif→adaptatif nécessite l'obfuscation indiscernable (iO). Le résultat d'Agrawal et al. l'évite grâce à la structure *uniforme* des MT : le même programme, avec temps variable, permet une Punctured Programming plus efficace qu'avec des circuits.

6 État de l'art

6.1 Tableau récapitulatif

Résultat	Année	Fonctions	Sécurité	Hypothèses	Collusion
Abdalla-CF	2015	$\langle \cdot, \cdot \rangle$	Adaptive	DDH	Non bornée
Gorbunov-VW	2012	Circuits	Sélective	LWE	Bornée
Ananth-V	2021	MT	Sélective	LWE	Bornée
Agrawal et al.	2022	MT	Adaptive	LWE+IBE+FHE	Bornée
Registered	2025	MT	Adaptive	LWE	Sans autorité

6.2 Registered FE pour MT (2025)

Le FE classique nécessite une autorité centrale (*Key Authority*) qui génère toutes les clés fonctionnelles — point de confiance unique.

Définition 6.1 (Registered FE). Dans un Registered FE, chaque utilisateur i génère (pk_i, sk_i) , enregistre pk_i auprès d'un helper public (non trusté), et obtient ses clés fonctionnelles sans exposer sk_i . Pas d'autorité centrale.

Théorème 6.2 (IACR ePrint 2025/967). *Il existe un Registered FE pour MT depuis LWE, sans autorité centrale.*

6.3 Frontières ouvertes

FE succinct. Construire un FE pour MT avec $|c| = \text{poly}(\lambda)$ indépendant de $T_M(|x|)$ depuis des hypothèses standards (sans iO) est un problème ouvert.

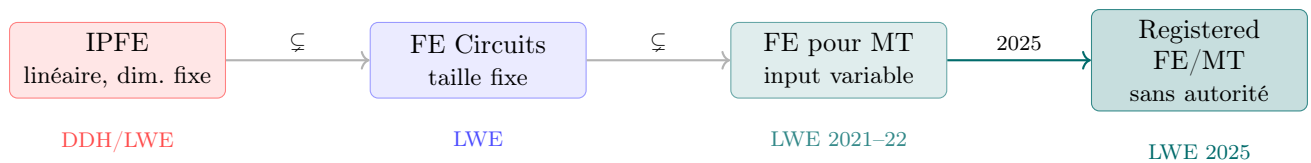
Efficacité. Les paramètres LWE actuels ($n \approx 1024\text{--}4096$, $q \approx 2^{100}$) rendent ces schémas non déployables. La réduction via module-LWE et réseaux algébriques est une direction active.

Collusion non bornée. La sécurité pour une collusion arbitraire nécessite iO. La réalisation de iO depuis des hypothèses standards (Jain-Lin-Sahai 2021 : LWE+LPN+DDH) est connue mais la praticabilité reste ouverte.

7 Synthèse

Fil directeur. L’IPFE calcule $\langle \mathbf{x}, \mathbf{y} \rangle$ pour une dimension n fixée. Cette contrainte est algébriquement fondamentale (Th. 3.1). Les Machines de Turing ont une description constante et s’adaptent à tout input. Le FE pour MT (Déf. 2.9) hérite de cette propriété : la même clé sk_M fonctionne pour tout x .

La construction d’Ananth-Vaikuntanathan (Th. 4.1) réalise ce schéma depuis LWE (post-quantique, fondé par la réduction de Regev au pire cas vers GapSVP), avec sécurité sélective et collusion bornée. Agrawal et al. (Th. 5.1) élèvent la sécurité au modèle adaptatif via Punctured Programming, sans iO.



Références

1. P. Ananth, V. Vaikuntanathan. *Optimal Bounded-Collusion Secure Functional Encryption*. CRYPTO 2021. https://dl.acm.org/doi/abs/10.1007/978-3-030-84259-8_9
2. S. Agrawal, E. Feng, O. Sarkar, D. Yadav. *Adaptively Secure Functional Encryption for Turing Machines*. ASIACRYPT 2022. https://dl.acm.org/doi/10.1007/978-3-031-22318-1_22
3. M. Abdalla, F. Bourse, A. De Caro, D. Pointcheval. *Simple Functional Encryption Schemes for Inner Products*. PKC 2015. (Schéma DDH-based IPFE, base de `ipfe_limit.py`.)
4. O. Regev. *On Lattices, Learning with Errors, Random Linear Codes, and Cryptography*. STOC 2005 (JACM 56(6), 2009). (Réduction GapSVP $\rightarrow$ LWE, Théorème 2.13.)
5. C. Gentry, C. Peikert, V. Vaikuntanathan. *Trapdoors for Hard Lattices*. STOC 2008.
6. R. Lu, R. Ostrovsky. *Garbled RAM Revisited*. EUROCRYPT 2013.
7. S. Gorbunov, V. Vaikuntanathan, H. Wee. *Functional Encryption with Bounded Collusions*. CRYPTO 2012. (FE pour circuits, base de `circuit_vs_tm.py`.)
8. F. Bourse. *Chiffrement Fonctionnel*. Thèse ENS Paris, 2017. <https://www.di.ens.fr/~fbourse/Thesis.pdf>
9. D. Stehlé. *IPFE — Notes ENS Lyon*. <https://perso.ens-lyon.fr/damien.stehle/IPFE.html>
10. IACR ePrint 2025/967. *Registered FE for Turing Machines from Lattices*. <https://eprint.iacr.org/2025/967>